EcoSphere Hackathon | Team: Winners4

Role: Voice Pipeline Engineer (Harsh Kalbande)

This single file contains everything you need to understand, run, and test the voice layer of EcoSphere. Forward this to any teammate who needs context on the audio pipeline.

Table of Contents

Role Overview

You own the audio layer end to end -- everything between the customer's microphone and the backend that runs the agent's brain.

You ARE responsible for:

- Agora RTC setup (app credentials, channel management)
- ASR (speech-to-text) configuration
- TTS (text-to-speech) configuration
- VAD (voice activity detection) and barge-in/interruption handling
- Connecting audio to the backend LLM endpoint

You are NOT responsible for:

- What the agent says -> that's the LangGraph/Agent Brain role
- Where product/pricing data comes from -> RAG role
- The webhook server code -> Backend/Frontend role

Architecture

```
Customer speaks
    ?
    ?
??????????????????????????
?  Agora RTC (audio)   ?  <- Low-latency audio transport
?  + Noise Suppression ?  <- Built-in echo cancellation
??????????????????????????
           ?
           ?
??????????????????????????
?  Agora ConvoAI       ?  <- Pipeline orchestrator
?  Engine              ?
??????????????????????????
           ?
     ?????????????????
     ?            ?
??????????????  ??????????????
? Deepgram?  ? MiniMax ?
? STT     ?  ? TTS     ?
? Nova-3  ?  ? Turbo   ?
??????????????  ??????????????
     ?            ?
     ?            ?
??????????????????????????
?  FastAPI Server      ?  <- OpenAI-compatible /chat/completions
?  (server/main.py)    ?
??????????????????????????
           ?
           ?
??????????????????????????
?  LangGraph Agent     ?  <- Intent routing, RAG, objection handling
?  (agent/graph.py)    ?
??????????????????????????
           ?
           ?
     Response text -> TTS -> Customer hears it
```

Prerequisites & Setup

Environment Variables (.env)

```
AGORA_APP_ID=66d8b08dbc7f47aa8abeed25ee9b02f4
AGORA_APP_CERTIFICATE=1e3fbbb8edf549adb224117bb3e87290
LLM_SERVER_URL=http://localhost:8000
```

!? For production/testing: Agora's cloud engine cannot reach localhost. You must run ngrok http 8000 and set LLM_SERVER_URL to the ngrok HTTPS URL.

Install Dependencies

```
pip3 install agora-agents langgraph langchain langchain-groq fastapi uvicorn
pip3 install python-dotenv httpx agora-token-builder
```

Running

```
# Terminal 1 -- FastAPI server
python3 server/main.py

# Terminal 2 -- Voice pipeline
python3 voice_pipeline.py

# Terminal 3 -- Expose server to Agora cloud
ngrok http 8000
# Then paste the https://... URL into .env as LLM_SERVER_URL
```

Step-by-Step Guide

Step 1 -- Get Agora project credentials

- [x] Project created at Agora Console
- [x] App ID: 66d8b08dbc7f47aa8abeed25ee9b02f4
- [x] App Certificate: stored in .env
- [x] Credentials stored securely, never committed to public repo

Step 2 -- Enable Conversational AI Engine

- [ ] Enable Conversational AI Engine in Agora Console (manual step)
- [x] ASR vendor: Deepgram (model: nova-3) -- fast, accurate, good English support
- [x] TTS vendor: MiniMax Turbo (model: speech_2_6_turbo) -- natural-sounding female voice

Step 3 -- Configure the LLM endpoint connection

- [x] Backend webhook URL: http://localhost:8000/chat/completions (or ngrok URL)
- [x] Request format: OpenAI Chat Completions ({ "model": "...", "messages": [...], "stream": true })
- [x] Response format: OpenAI SSE streaming ({ "choices": [{ "delta": { "content": "..." } }] })

Step 4 -- Build a minimal test app

- [x] test_call.html -- minimal join/speak UI
- [x] demo.html -- full dashboard with transcript + analytics

Step 5 -- Test interruption / barge-in behavior

- [x] test_barge_in.py created for manual interruption testing
- [ ] Run manual test: join call -> let agent speak -> interrupt by talking
- [ ] Confirm TTS stops immediately when interruption detected

Step 6 -- Noise handling check

- [x] Agora RTC SDK has built-in noise suppression + echo cancellation
- [ ] Manual test: run in moderately noisy environment

Step 7 -- Full pipeline test

- [ ] All components wired; needs ngrok + Agora Console enabled
- [ ] Measure round-trip latency

voice_pipeline.py

```
import os
import time
from dotenv import load_dotenv
from agora_agent import Agent, Agora, Area, DeepgramSTT, CustomLLM, MiniMaxTTS

load_dotenv()

# Server URL - use ngrok URL for Agora (cloud can't reach localhost)
# Start ngrok: ngrok http 8000
# Then paste the https URL below or in .env as LLM_SERVER_URL
LLM_SERVER_URL = os.getenv("LLM_SERVER_URL", "http://localhost:8000")

AGENT_PROMPT = """You are a friendly AI sales assistant.
Qualify the lead: understand their needs, budget, and timeline.
Keep responses short and conversational."""

GREETING = "Hi! I'm your AI sales assistant. How can I help you today?"

AGORA_APP_ID = os.getenv("AGORA_APP_ID")
AGORA_APP_CERT = os.getenv("AGORA_APP_CERTIFICATE")
if not AGORA_APP_ID or not AGORA_APP_CERT:
    raise ValueError("AGORA_APP_ID and AGORA_APP_CERTIFICATE must be set in .env")

client = Agora(area=Area.US, app_id=AGORA_APP_ID,
               app_certificate=AGORA_APP_CERT)

# LangGraph agent via FastAPI server (run server first: python3 server/main.py)
agent = (
    Agent(client=client, turn_detection={"language": "en-US"})
    .with_stt(DeepgramSTT(model="nova-3", language="en"))
    .with_llm(CustomLLM(
        api_key="not-needed",
        base_url=f"{LLM_SERVER_URL}/chat/completions",
        model="ecosphere-sales-agent",
        system_messages=[{"role": "system", "content": AGENT_PROMPT}],
        greeting_message=GREETING,
        max_history=50,
        params={"max_tokens": 256, "temperature": 0.7}))
    .with_tts(MiniMaxTTS(model="speech_2_6_turbo",
        voice_id="English_captivating_female1"))
)

channel_name = f"sales-call-{int(time.time())}"

session = agent.create_session(
    channel=channel_name,
    agent_uid="100001", remote_uids=["*"],
    name=f"conversation-{int(time.time())}",
    idle_timeout=60, debug=True)

print("=" * 50)
print(f"? CHANNEL NAME: {channel_name}")
print("=" * 50)
print(" Copy the channel name above and paste it in test_call.html")
print("? Voice pipeline started! Press Ctrl+C to stop.")
```

```
session.start()
```

test_barge_in.py

```
"""
EcoSphere - Barge-In / Interruption Test Script
================================================
Tests that the voice pipeline correctly handles interruptions:
when the customer speaks while the agent is talking, the TTS should
stop immediately and the agent should listen to the new input.

How to use:
1. Start the server: python3 server/main.py
2. Start the voice pipeline: python3 voice_pipeline.py
3. Open test_call.html, paste the channel name, and join
4. When the agent starts speaking, interrupt by talking
5. Watch the terminal output for interruption detection

What to verify:
- Agent TTS stops immediately when you speak
- Your new speech is transcribed (ASR picks it up)
- Agent responds to your interruption, not the old message
- No audio overlap (agent and customer not talking simultaneously)

VAD Tuning Notes:
- Default VAD in Agora ConvoAI should work for most cases
- If interruptions are too eager (cutting off normal pauses):
  -> Increase silence threshold in Agora Console settings
- If interruptions are too slow (not detecting real ones):
  -> Decrease silence threshold or increase sensitivity
- Key setting: "Minimum speech duration" controls how long you
  must speak before an interruption is recognized
"""

import os
import sys
import time
import json
from dotenv import load_dotenv

load_dotenv()

AGORA_APP_ID = os.getenv("AGORA_APP_ID")
AGORA_APP_CERT = os.getenv("AGORA_APP_CERTIFICATE")

if not AGORA_APP_ID or not AGORA_APP_CERT:
    print("[X] AGORA_APP_ID and AGORA_APP_CERTIFICATE must be set in .env")
    sys.exit(1)

try:
    from agora_agent import Agent, Agora, Area, DeepgramSTT, CustomLLM, MiniMaxTTS
except ImportError:
    print("[X] agora-agent SDK not installed. Run: pip3 install agora-agents")
    sys.exit(1)

LLM_SERVER_URL = os.getenv("LLM_SERVER_URL", "http://localhost:8000")

AGENT_PROMPT = """You are a friendly AI sales assistant.
Qualify the lead: understand their needs, budget, and timeline.
```

```
Keep responses short and conversational.
IMPORTANT: If the customer interrupts you, stop talking immediately and respond to what they...

GREETING = "Hi! I'm your AI sales assistant. I'm here to help you learn about our product. H...

client = Agora(area=Area.US, app_id=AGORA_APP_ID, app_certificate=AGORA_APP_CERT)

agent = (
    Agent(client=client, turn_detection={"language": "en-US"})
    .with_stt(DeepgramSTT(model="nova-3", language="en"))
    .with_llm(CustomLLM(
        api_key="not-needed",
        base_url=f"{LLM_SERVER_URL}/chat/completions",
        model="ecosphere-sales-agent",
        system_messages=[{"role": "system", "content": AGENT_PROMPT}],
        greeting_message=GREETING,
        max_history=50,
        params={"max_tokens": 256, "temperature": 0.7}))
    .with_tts(MiniMaxTTS(model="speech_2_6_turbo",
        voice_id="English_captivating_female1"))
)

channel_name = f"barge-in-test-{int(time.time())}"

print("=" * 60)
print("?  BARGE-IN / INTERRUPTION TEST")
print("=" * 60)
print(f"[sat] Channel: {channel_name}")
print()
print("INSTRUCTIONS:")
print("  1. Open test_call.html in your browser")
print(f"  2. Paste channel name: {channel_name}")
print("  3. Click 'Join Call'")
print("  4. Let the agent start speaking, then INTERRUPT by talking")
print("  5. Verify the agent stops and responds to your interruption")
print()
print("What to watch for in terminal output:")
print("  - '? Interruption detected' or similar when you speak over agent")
print("  - Your interrupted speech is transcribed")
print("  - Agent responds to your new input")
print()
print("Press Ctrl+C to stop the test.")
print("=" * 60)

session = agent.create_session(
    channel=channel_name,
    agent_uid="100001",
    remote_uids=["*"],
    name=f"barge-in-test-{int(time.time())}",
    idle_timeout=60,
    debug=True  # Enable debug logging to see interruption events
)

session.start()
```

test_call.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Winners4 - Test Call</title>
  <script src="https://download.agora.io/sdk/release/AgoraRTC_N.js"></script>
  <style>
    body { font-family: Arial; background: #1a1a2e; color: #fff; display: flex; justify-cont...
    .box { background: #16213e; padding: 40px; border-radius: 16px; text-align: center; widt...
    h1 { color: #e94560; margin-bottom: 20px; }
    input { width: 100%; padding: 12px; margin: 8px 0; border: none; border-radius: 8px; bac...
    button { width: 100%; padding: 14px; margin: 12px 0; border: none; border-radius: 8px; f...
    #joinBtn { background: #e94560; color: #fff; }
    #leaveBtn { background: #533483; color: #fff; }
    .status { margin-top: 16px; padding: 12px; border-radius: 8px; background: #0f3460; }
    .ok { background: #1b5e20; }
    .err { background: #b71c1c; }
  </style>
</head>
<body>
  <div class="box">
    <h1>? Winners4 Voice Test</h1>
    <input id="channel" placeholder="Paste channel name from terminal">
    <button id="joinBtn" onclick="join()">? Join Call</button>
    <button id="leaveBtn" onclick="leave()" style="display:none">? Leave</button>
    <div class="status" id="status">Paste channel name from terminal, then click Join</div>
  </div>
  <script>
    let client, mic;
    async function join() {
      const ch = document.getElementById('channel').value;
      if (!ch) { show('Enter channel name','err'); return; }
      client = AgoraRTC.createClient({mode:'rtc',codec:'vp8'});
      mic = await AgoraRTC.createMicrophoneAudioTrack();
      client.on('user-published', async (u, t) => {
        await client.subscribe(u, t);
        if (t==='audio') { u.audioTrack.play(); show('[vol] AI speaking -- talk now!','ok'); }
      });
      try {
        await client.join('66d8b08dbc7f47aa8abeed25ee9b02f4', ch, null, null);
        await client.publish([mic]);
        document.getElementById('joinBtn').style.display='none';
        document.getElementById('leaveBtn').style.display='block';
        show('[OK] Connected! Start speaking...','ok');
      } catch(e) { show('Error: '+e.message,'err'); }
    }
    async function leave() {
      if(mic) mic.close();
      if(client) await client.leave();
      document.getElementById('joinBtn').style.display='block';
      document.getElementById('leaveBtn').style.display='none';
      show('Left call','');
    }
    function show(m,t){ const e=document.getElementById('status'); e.textContent=m; e.classN...
  </script>
```

```
</body>
</html>
```

Handoff Checklist

What other roles need from the Voice Pipeline:

- [x] Working Agora App ID + Certificate -- stored in .env
- [x] Confirmed request/response JSON contract -- OpenAI Chat Completions format at /chat/completions
- [x] Test call script/demo -- test_call.html + test_barge_in.py
- [x] Documented ASR/TTS vendor choices -- Deepgram Nova-3 (STT) + MiniMax Turbo (TTS)

What still needs manual action:

1. Enable ConvoAI Engine in Agora Console (can't be done from code)
2. Start ngrok and update LLM_SERVER_URL in .env (Agora cloud can't reach localhost)
3. Run barge-in test manually to verify interruption handling
4. Test with background noise to confirm noise suppression

Common Pitfalls

1. Forgetting barge-in testing -- Easy to build a pipeline that works for simple Q&A but breaks the moment someone interrupts. Use test_barge_in.py.
2. Mismatched JSON contract -- The server expects OpenAI Chat Completions format. Confirm with Person 4 (backend) that the request/response format matches before testing end-to-end.
3. Testing only in silence -- Real demo conditions (stage mic, audience noise) won't be silent. Always test with background noise.
4. ngrok URL expiring -- Free ngrok URLs change on restart. If Agora calls suddenly fail, check if ngrok is still running and if LLM_SERVER_URL is still valid.
5. Credentials hardcoded -- Never hardcode AGORA_APP_ID or AGORA_APP_CERTIFICATE in source files. Always use .env.

Generated by Codebuff ? for Winners4 -- EchoSphere Hackathon